

Oui. Voici la liste consolidée des améliorations identifiées pendant les développements.

Améliorations fonctionnelles

Navigation et contexte

- Généraliser le paramètre `next` pour toutes les transactions.
- Distinguer systématiquement :
 - navigation globale depuis le sidebar ;
 - navigation contextualisée depuis un projet, un lot ou une réunion.
- Ajouter progressivement un fil d'Ariane.
- Enrichir la TopBar avec le contexte courant :
 - projet ;
 - lot ;
 - éventuellement tâche ou réunion.
- Conserver le workspace projet comme pivot principal.

Listes et filtres

- Pouvoir masquer les colonnes inutiles selon le contexte.
 - Exemple : ne pas afficher « Projet » dans une liste déjà filtrée par projet.
- Permettre à l'utilisateur de choisir :
 - colonnes visibles ;
 - ordre des colonnes ;
 - tri par défaut ;
 - taille de page ;
 - filtres par défaut.
- Introduire des vues enregistrées :
 - réunions à venir ;
 - dernières réunions terminées ;
 - risques actifs ;
 - tâches en retard ;
 - mes éléments.
- Distinguer :
 - définition générale de la liste ;
 - vue utilisateur persistante ;
 - filtre ponctuel.
- Ne pas figer arbitrairement des horizons comme « deux mois ».

Réunions

- Remplacer la gestion CRUD des participants par un écran unique avec formsets.
- Séparer visuellement :
 - participants internes ;
 - participants externes.
- Ajouter ou supprimer plusieurs participants sans quitter la réunion.
- Garder la réponse à l'invitation facultative.
- Prévoir une automatisation légère future des réponses reçues.
- Prévoir une interface par onglets :
 - informations générales ;
 - participants ;
 - ordre du jour ;
 - compte-rendu ;
 - pièces jointes.
- Ne pas imposer de compte-rendu.
- Laisser les documents produits par une réunion au module Documentation.

Documentation

- Séparer l'objet métier `Document` du fichier physique.
- Gérer les versions comme des enfants du document.
- Utiliser un catalogue incrémental pour les types documentaires.
- Gérer les capacités selon le format et les outils disponibles :
 - édition ;
 - aperçu ;
 - signature ;
 - impression ;
 - indexation ;
 - analyse IA.
- Pouvoir associer un document à :
 - projet ;
 - lot ;
 - tâche ;
 - risque ;
 - réunion.
- Distinguer clairement :
 - téléverser : intégrer un fichier externe ;
 - télécharger : récupérer un fichier stocké.
- Prévoir des liens entre documents.
- Concevoir le DOE comme un processus :
 - sélection de documents ;

- proposition d'arborescence par IA ;
- validation par l'utilisateur.
- Étudier le cycle de vie documentaire avant le développement.

Équipes et affectations

- Décider du modèle entre :
 - affectation individuelle ;
 - équipes permanentes ou temporaires ;
 - modèle mixte.
- Séparer :
 - prévision ;
 - organisation quotidienne ;
 - réel remonté par les rapports d'activité.
- Éviter le micro-pilotage horaire des personnes.
- Prévoir les sorties temporaires d'un membre pour une urgence.
- Conserver l'autonomie du chef d'équipe.
- Ne pas commencer le planning détaillé avant cette décision.

Planning

- Afficher le planning d'un lot.
- Ajouter ensuite l'accès planning depuis le projet.
- Préremplir éventuellement certaines dates à partir du parent, sans jamais écraser la décision de l'utilisateur.
- Ne pas déduire automatiquement une durée de tâche uniquement depuis la charge tant que les équipes ne sont pas modélisées.
- Prévoir un accès direct aux tâches depuis le planning.

Risques

- Ajouter ultérieurement une criticité proposée, mais non imposée.
- Étudier un niveau de surveillance distinct de la criticité.
- Prévoir des vues :
 - actifs ;
 - critiques ;
 - non revus récemment ;
 - clos.
- Permettre qu'une réunion soit à l'origine d'un risque sans créer de dépendance forte entre modules.

Améliorations techniques

Framework

- Créer un mixin générique de gestion des URLs de retour.
- Factoriser la construction des listes dans une `EPListView`.
- Étudier une déclaration standard de module, par exemple `EPModule`.
- Supporter les colonnes contextuelles.
- Ajouter un mécanisme générique de filtres et vues enregistrées.
- Prévoir les préférences utilisateur au niveau du framework.
- Réduire la duplication de :
 - `get_return_url()` ;
 - `get_success_url()` ;
 - `get_cancel_url()` ;
 - construction de `EPList` et `ListPage`.

Références métier

- Adopter `reference` pour les nouvelles entités.
- Prévoir une transaction dédiée pour renommer progressivement `code` en `reference`.
- Faire évoluer le générateur afin qu'il accepte un nom de champ paramétrable.
- Renommer ultérieurement `normalize_code_part()` de manière plus générique.

Formulaires

- Factoriser la configuration des champs catalogue.
- Factoriser l'application des valeurs par défaut.
- Introduire des formsets pour les collections éditées sur un seul écran.
- Maintenir des formulaires simples, avec le minimum de champs obligatoires.

Architecture métier

- Un module reste responsable de ses propres objets.
- Les interactions se font par associations.
- Une réunion peut être la source d'un document, d'un risque ou d'une tâche, mais ne gère pas directement leur métier.
- L'outil propose et alerte ; il ne décide pas à la place de l'utilisateur.
- Distinguer données essentielles et données de gouvernance.

Scénarios de test

Vous avez raison : cela devient un chantier structurant.

Je proposerais de construire un **jeu de données de référence stable**, puis des scénarios métier qui s'appuient dessus.

Socle de données

Sociétés

Prévoir au minimum :

- société utilisatrice principale ;
- maître d'ouvrage ;
- maître d'œuvre ;
- sous-traitant ;
- fournisseur ;
- société inactive.

Utilisateurs

Créer plusieurs profils :

- administrateur système ;
- administrateur client ;
- chef de projet ;
- conducteur de travaux ;
- chef d'équipe ;
- compagnon ;
- sous-traitant ;
- utilisateur en lecture seule ;
- utilisateur inactif.

Prévoir des utilisateurs ayant plusieurs rôles selon les projets.

Projets

Créer des projets représentatifs :

- projet planifié sans lot ;
- projet actif simple ;
- projet actif complexe ;
- projet suspendu ;
- projet terminé ;
- projet sans dates ;
- projet avec dates révisées ;
- projet sans chef de projet ;
- projet avec beaucoup de données.

Lots

Pour chaque projet :

- aucun lot ;
- un seul lot ;
- plusieurs lots ;
- lot sans responsable ;
- lot sans dates ;
- lot suspendu ;
- lot terminé ;
- références automatiques et manuelles.

Tâches

Prévoir :

- tâche planifiée ;
- en cours ;
- suspendue ;
- terminée ;
- sans dates ;
- avec dates révisées ;
- sans charge ;
- avec avancement limite à 0 % et 100 % ;
- référence automatique et manuelle.

Risques

Prévoir :

- risque latent ;
- actif ;
- clos ;
- opportunité ;
- faible criticité ;
- critique ;
- sans pilote ;
- sans coût ;
- avec dates incohérentes pour les tests de validation.

Réunions

Prévoir :

- réunion future ;
- réunion passée ;
- terminée ;

- annulée ;
- sans durée ;
- avec notes publiques ;
- avec commentaires internes ;
- participants internes uniquement ;
- externes uniquement ;
- mixte ;
- réponses absentes, acceptées et refusées.

Types de tests

Tests unitaires

- validations de modèles ;
- génération des références ;
- unicité dans le périmètre du parent ;
- normalisation ;
- propriétés calculées ;
- règles interne/externe des participants.

Tests d'intégration

- société → utilisateur → projet ;
- projet → lot → tâche ;
- projet → risque ;
- projet → réunion → participant ;
- catalogues et valeurs par défaut ;
- suppression protégée par les relations.

Tests de navigation

- liste globale → modifier → retour global ;
- workspace → transaction contextualisée → retour workspace ;
- lot → tâches → retour lots ;
- réunion → participants → retour réunions ;
- création et annulation avec `next` .

Tests fonctionnels

- création d'un projet complet ;
- lotissement ;
- création des tâches ;
- déclaration d'un risque ;
- organisation d'une réunion ;
- ajout de participants internes et externes ;

- consultation du workspace ;
- modification successive des éléments.

Tests de volumétrie

Prévoir un jeu plus lourd :

- 20 projets ;
- 10 lots par projet ;
- 50 tâches par lot ;
- 30 risques par projet ;
- 100 réunions par projet ;
- plusieurs milliers de participants.

Cela permettra de tester les listes, les tris, les requêtes `select_related` et la pagination.

Organisation recommandée

Je créerais trois jeux de données :

minimal

Un élément de chaque type, utilisé pour les tests rapides.

functional

Des données réalistes couvrant les scénarios métier.

volume

Un jeu généré automatiquement pour les performances et la pagination.

Le sujet des équipes devra être traité avant de stabiliser définitivement les scénarios de planning et de rapports d'activité, mais le reste peut déjà être structuré.